B.Sc. (ECS)-II

Advanced Python Unit-2

Unit 2 contains Advanced file handling, working with json file, Working with CSV Files, Working with excel file.

Advanced file handling

We know that a file is a place on secondary storage where data can be stored permanently. We learned basic file operations previously like writing and appending text or binary as Now we are going to learn advanced file handling.

Use of with statement:

A common problem occurs when working with files is that though we call obj.close() To close the file handler, some times that file handler remains open because of occurrence of an exception in previous statement.
The solution for this problem is that we can put obj.close() statement in finally block or we can use another way to close the file handler compulsory by with statement at the time of file opening.

The with statement in Python helps us to resource management. It ensures we don't accidentally leave any file handler open. It calls __exit()__ *method to close handler automatically at the end.*

The with statement is a replacement for commonly used try-finally error-handling statements.
Syntax

```
with open("filename", "mode") as obj:
```

Example: File writing operation

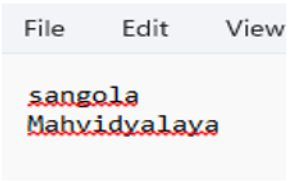
```
with open('d:/demo.txt','w') as f:
    f.write("sangola")
print("operation successful")
```

Example: File Reading operation

```
with open('d:/demo.txt','r') as f:
    s=f.read()
print(s)
```

We can also loop through the file and print the text line by line:

```
with open('d:/demo.txt','r') as f:
    for x in f:
        print(x)
```



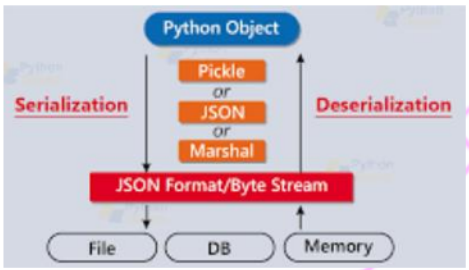

```
sangola

Mahvidyalaya
```

Serialization and Deserialization

Serialization and deserialization are the processes used to convert complex data structures, such as objects or data collections, into a format that can be easily stored or transmitted and it allows us to convert data back into its original form.

We can use any one of below three modules for serialization and deserialization

1) pickle 2) json 3) marshal



Serialization:

Serialization is the process of converting an object's state to a byte stream or a string representation which can be easily stored or transmitted.

Serialization is also known as pickling. dump(data,obj) method used for serialization.

Example of serialization by pickle module

```
import pickle
data={'name':'abc','place':'sangola'}
with open("d:/demo.bin","wb")as f:
    pickle.dump(data,f)
```

Deserialization:

Deserialization is the reverse process of converting byte stream into object type.

Deserialization is also known as Unpickling. load(obj) method used for deserialization.

Example of deserialization by pickle module

```
import pickle
with open("d:/demo.bin","rb")as f:
    x=pickle.load(f)

print(x)
```

Output:

{'name': 'abc', 'place': 'sangola'}

working with json file

JSON (JavaScript Object Notation) is a script (executable) file which is made of text and it is used to store and transfer the data.

Python supports JSON through a built-in package called **json**.

We can convert Python data types to a JSON-formatted string with json.dumps() or write them to **files** using json.dump().

Similarly, we can and parse JSON strings with json.loads() and read JSON data from **files** with json.load()

Convert dict to json string:

To convert a Python dictionary to JSON, the built-in json module is used.

The json.dumps() method converts a Python dictionary into a JSON-formatted string.

This is useful when we need to send the JSON data over a network, store it as a string, or print it for inspection.

Syntax:

```
json.dumps(data,indent)
```

Example:

```
import json
d={
   "id":101,
   "name": "Abc",
   "place":"Sangola"
  }
r=json.dumps(d,indent=4)
print(r)
print(type(r))
```

Output:

```
{
   "id": 101,
   "name": "Abc",
   "place": "Sangola"
}
<class 'str'>
```

Convert dict to json file (Serialization using JSON):

To convert dict to json file, the json.dump() method is used.

This method writes a dictionary directly into a file in JSON format.

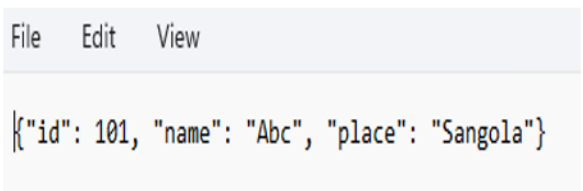
Syntax:

```
json.dump(data,fileobj)
```

Example:

```
import json
d={
   "id":101,
   "name": "Abc",
   "place":"Sangola"
  }
with open("d:/new.json","w")as f:
    json.dump(d,f)
```

Output:



Convert json string to dict:

We can convert a json string into dictionary, for that the json module's **loads()** method is used.

Syntax:

```
json.loads(data)
```

Example:

```
import json
d='{"id":"101","name": "Abc","place":"Sangola"}'
js=json.loads(d)
print(js)
print(type(js))
```

```
{'id': '101', 'name': 'Abc', 'place': 'Sangola'}
<class 'dict'>
```

Convert json file to dict (Deserialization using JSON):

Deserialization is the opposite of Serialization, i.e. conversion of JSON objects into their respective Python objects. The **load()** method is used for it.

Syntax:

```
json.load(fileobj)
```

Example:

```
import json
with open("d:/new.json","r")as f:
    r=json.load(f)
print(r)
print(type(r))
```

```
{'id': 101, 'name': 'Abc', 'place': 'Sangola'}
<class 'dict'>
```

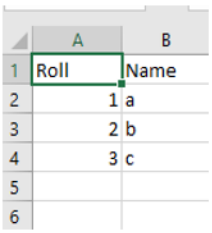
Working with CSV Files

The CSV (Comma Separated Values) format is a common and straightforward way to store tabular data. The CSV file should have the .csv file extension.
Because it is a plain text file, it can contain only actual text data.
Normally, CSV files use a comma to separate each data value.
Python provides **csv** module to work with csv files. The module includes various methods to perform different operations.

reading csv file with csv module:

Reading from a CSV file is done using the reader object. The CSV file is opened as a text file with Python's built-in open() function, which returns a file object. This is then passed to the reader().
Example: Consider we have a csv file named democs.csv

```
import csv
with open('D:/democs.csv', 'r') as f:
    r = csv.reader(f)

    # extracts the first row as column headers.
    header = next(r)
    print("Header:", header)

    # Iterate and print each data row
    for row in r:
        print("Data Row:", row)
```



	A	B
1	Roll	Name
2	1	a
3	2	b
4	3	c
5		
6		

```
Header: ['Roll ', 'Name']
Data Row: ['1', 'a']
Data Row: ['2', 'b']
Data Row: ['3', 'c']
```

Another Example

```
import csv
fields = []  # Column names
rows = []    # Data rows
with open('D:/democs.csv', 'r') as f:
    r = csv.reader(f)  # Reader object
    fields = next(r)  # Read header
    for row in r:     # Read rows
        rows.append(row)
    print("Total no. of rows: %d" % r.line_num)  # Row count
print('Field names are: ' + ', '.join(fields))
print('\nFirst 2 rows are:\n')
for row in rows[:2]:
    for col in row:
        print("%10s" % col, end=" ")
    print('\n')
```

Reading CSV files into a dictionary with csv module:

We can read CSV data directly into a dictionary by using csv.DictReader() method .
Also we have to use data_list.append(row) method to stores each dictionary in a list.

```
import csv
d=[]
with open('D:/democs.csv', 'r') as f:
    r = csv.DictReader(f)
    for row in r:
        d.append(row)
print(d)
```

Output:

```
[{'Roll ': '1', 'Name': 'a'}, {'Roll ': '2', 'Name': 'b'}, {'Roll ': '3', 'Name': 'c'}]
```

writing CSV files with csv module:

The csv.writer() is used to write data directly to a CSV file. We can also write a single row to by using write_row() method to write multiple rows.

```
import csv
data = [
    ['Roll','Name'],
    [1, 'a'],
    [2, 'b'],
    [3, 'c']
      ]
with open('D:/democs', 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerows(data)
```

Writing CSV file from a dictionary with csv module/Writing a dictionary to a CSV file

To write a dictionary to a CSV file the csv.DictWriter () is used. It maps dictionary keys to CSV column headers, simplifying the process of writing structured data.

```
import csv
data = [
    {"empid":101,"Name": "ABC"},
    {"empid":102,"Name": "PQR"}
      ]

fields = ["empid","Name" ]

with open("D:/new.csv", 'w', newline='') as f:
    writer = csv.DictWriter(f,
fieldnames=fields)
    writer.writeheader()
    writer.writerows(data)
```

Working with excel file

Xlsx files are the most widely used documents in the technology field by data scientists. The openpyxl library in Python enables interaction with Excel files, specifically those in the .xlsx format. This includes reading, writing, and modifying data within spreadsheets.
To work with excel first we have to install openpyxl module by cmd as

```
pip install openpyxl
```

Now lets see operation performed on excel by using python

Basic Excel Terminologies:

Workbook: A spreadsheet is represented as a workbook in openpyxl. A workbook consists of one or more sheets.

Sheet: A sheet is a single page composed of cells for organizing data.

Cell: The intersection of a row and a column is called a cell. Usually represented by A1, B5, etc.

Row: A row is a horizontal line represented by a number (1,2, etc.).

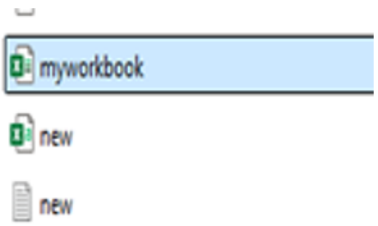
Column: A column is a vertical line represented by a capital letter (A, B, etc.).

Creating and write a Simple Spreadsheet:

A workbook is essentially the Excel file itself. We can use the Workbook () from openpyxl to create an Workbook instance i.e., an Excel file.

```
from openpyxl import Workbook

# Create a workbook
wb = Workbook()

# Get the active worksheet or create a new sheet
ws = wb.active

# Optionally, you can rename the default sheet
ws.title = "MySheet"

# Save the workbook to a file
wb.save("d:/myworkbook.xlsx")
```

Output:

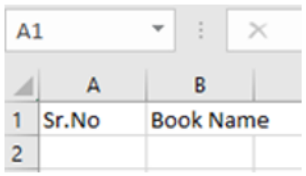


Loading workbook and Adding values to cell

load_workbook() method used to load/reopen existing workbook.

```
from openpyxl import load_workbook
# load a workbook
wb = load_workbook('d:/myworkbook.xlsx')

# Get the active worksheet or create a new sheet
ws = wb.active
ws['A1']="Sr.No"
ws['B1']="Book Name"
# Save the workbook to a file
wb.save("d:/myworkbook.xlsx")
```

Output:

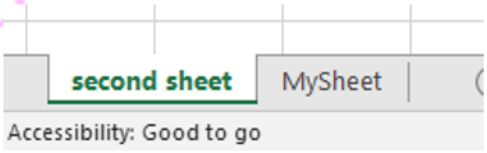


Note: Before execute above code close the excel sheet

Creating another Sheet in Workbook:

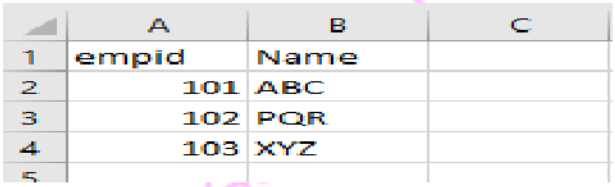
We can create a worksheet by using create_sheet() method

```
from openpyxl import load_workbook
# load a workbook
wb = load_workbook('d:/myworkbook.xlsx')
# Creates a sheet at the beginning
ws = wb.create_sheet("second sheet",0)
# Save the workbook to a file
wb.save("d:/myworkbook.xlsx")
```



Reading excel spreadsheets:

Consider a worksheet



	A	B	C
1	empid	Name	
2	101	ABC	
3	102	PQR	
4	103	XYZ	
5			

We can read excel sheet by different techniques.

a) Reading Data from a Particular Cell:

We can read specific cell either by obj[‘cell spec. ’].value or obj.cell(row=value,column=value).value

```
from openpyxl import load_workbook
wb = load_workbook('D:/new.xlsx')
ws=wb.active
x=ws['A2']
y=ws.cell(row=2,column=2)
print(x.value)
print(y.value)
```

Output:

```
=== RES
101
ABC
>>>
```

b) Read/Print only Column Headers

To read only columns Use max_column to iterate through all column (max_column returns total number of columns)

```
from openpyxl import load_workbook

wb = load_workbook('D:/new.xlsx')
ws=wb.active
for i in range(1,ws.max_column+1):
    print(ws.cell(row=1,column=i).value)
```

```
emp id
emp name
>>>
```

c) Read/Print All Values of the First Column

To list all entries from the first column (e.g. emp id). Use max_row to iterate through all rows in column 1.(max_row returns total rows in sheet)

```
from openpyxl import load_workbook

wb = load_workbook('D:/new.xlsx')
ws=wb.active
for i in range(1,ws.max_row+1):
    print(ws.cell(row=i,column=1).value)
```

Output:

```
emp id
101
102
103
>>>
```

Updating Cell Values:

Updating cell values in an Excel workbook using the openpyxl involves loading the workbook, accessing the desired sheet, and then assigning values to specific cells by referencing its coordinates (e.g., 'A1', 'B2') or by using the cell() method with row and column numbers.

```
from openpyxl import load_workbook

wb = load_workbook('D:/new.xlsx')
ws=wb.active  #ws=wb[sheetname]
ws['B4']="MNO" #wb.cell(row=4,column=2).value="MNO"
wb.save("d:/new.xlsx")
```

Before

emp id	emp name
101	ABC
102	PQR
103	XYZ

After

emp id	emp name
101	ABC
102	PQR
103	MNO

We can add new row values in previous sheet as

```
from openpyxl import load_workbook

wb = load_workbook('D:/new.xlsx')
ws=wb.active  #ws=wb[sheetname]
r= (104,'NNN')
ws.append(r)
```

Managing Rows and Columns

a) insert row:

To insert a row into an Excel we can use the insert_rows(idx,amount) method.

Where,

idx - row number before which we want to insert

amount(optional**)**- number of rows to insert, defaulting to 1.

```
from openpyxl import load_workbook
wb = load_workbook('D:/new.xlsx')
ws=wb.active  #ws=wb[sheetname]
ws.insert_rows(idx=3,amount=10)
wb.save("d:/new.xlsx")
```

b) delete row:

To delete a row from Excel we can use the delete_rows(idx,amount) method.

Where,

idx - row number from which we want to delete

amount(optional**)**- number of rows to delete, defaulting to 1.

```
from openpyxl import load_workbook
wb = load_workbook('D:/new.xlsx')
ws=wb.active  #ws=wb[sheetname]
ws.delete_rows(idx=3,amount=10)
wb.save("d:/new.xlsx")
```

c) insert column:

We can insert a column in worksheet by using insert_cols(idx,amount) method .

Where,

idx - column number from which we want to delete

amount(optional**)**- number of columns to insert, defaulting to 1.

```
from openpyxl import load_workbook
wb = load_workbook('D:/new.xlsx')
ws=wb.active  #ws=wb[sheetname]
ws.insert_cols(idx=2,amount=10)
wb.save("d:/new.xlsx")
```

d) delete column:

To delete a column from Excel we can use the delete_cols(idx,amount) method.

Where,

idx - column number from which we want to delete

amount(optional**)**- number of column to delete, defaulting to 1.

```
from openpyxl import load_workbook
wb = load_workbook('D:/new.xlsx')
ws=wb.active  #ws=wb[sheetname]
ws.delete_cols(idx=2,amount=10)
wb.save("d:/new.xlsx")
```

Managing Sheets:

The openpyxl offers functionalities for managing sheets within an Excel workbook as,

1. Loading and Accessing Sheets:

Loading a Workbook

```
from openpyxl import load_workbook
    wb = load_workbook('your_excel_file.xlsx')
```

Accessing the Active Sheet

```
ws = wb.active
```

Accessing a Sheet by Name

```
ws = wb['SheetName']
```

Getting All Sheet Names

```
    sheet_names = wb.sheetnames
```

2. Creating and Deleting Sheets:

Creating a New Sheet

```
new_sheet = wb.create_sheet("New Title", index=0) # index is optional
```

Deleting a Sheet

```
    del wb['SheetToDelete']
    # or
    wb.remove(sheet_object)
```

3. Manipulating Sheet Properties:

Renaming a Sheet

```
sheet.title = "New Title"
```

4. Iterating Through Sheets:

Looping through all sheets

```
for sheet in wb:
        print(sheet.title)
```

5. Saving the Workbook:

Saving Changes

```
wb.save('your_excel_file.xlsx')
```

Freezing Row:

To freeze a row in worksheet, we can use freeze_panes(cell) method.
This method takes a cell reference as a string, and it freezes all rows above and all columns to the left of the specified cell.

```
from openpyxl import load_workbook
wb = load_workbook('D:/new.xlsx')
ws=wb.active  #ws=wb[sheetname]
# Freeze the first row
ws.freeze_panes = 'A2'
wb.save('D:/new.xlsx')
```

Freezing Column:

To freeze a column in worksheet, we can use freeze_panes(cell) method.
It will freeze all rows above and all columns to the left of that specified cell.

Freezing a single column

```
from openpyxl import load_workbook
wb = load_workbook('D:/new.xlsx')
ws=wb.active  #ws=wb[sheetname]
ws.freeze_panes = 'B1'
wb.save('D:/new.xlsx')
```

This will freeze only one columns i.e. A

Freezing a Multiple columns

```
from openpyxl import load_workbook
wb = load_workbook('D:/new.xlsx')
ws=wb.active  #ws=wb[sheetname]
ws.freeze_panes = 'C1'
wb.save('D:/new.xlsx')
```

This will freeze two columns i.e. A and B

Adding Filters:

It's possible to filter single range of values in a worksheet.

If there is a need to filter multiple ranges, we can use tables and apply a separate filter for each table.

To add a filter we can define a range. Filters are then applied to columns in the range using a zero-based index, eg. in a range from A1:H10, colId 1 refers to column B.

Note Filters and sorts can only be configured by openpyxl but will need to be applied in applications like Excel.

```
from openpyxl import load_workbook
wb=load_workbook("D:/new.xlsx")
ws=wb.active
ws.auto_filter.ref="A1:B14"
ws.auto_filter.add_filter_column(1,["PQR","MNO"])
wb.save("D:/new.xlsx")
```

epm id	name
113	ABC
102	PQR
103	MNO

Adding Formulas:

We can add formulas in Excel by using OpenPyxl. We can assign a formula to a cell just like any other value.

IT involves assigning a string representing the Excel formula to the desired cell.

Consider a worksheet formula.xlsx

PHY.	CHEM.	MATHS	Total	Avg.
70	77	90		

```
from openpyxl import load_workbook
wb=load_workbook("D:/formula.xlsx")
ws=wb.active
#formula for sum
ws["E5"]="=SUM(B5,C5,D5)"
#formula for average
ws["F5"]="=AVERAGE(B5,C5,D5)"
wb.save("D:/formula.xlsx")
```

Output:

PHY.	CHEM.	MATHS	Total	Avg.
70	77	90	237	79

Another Example

```
from openpyxl import load_workbook
wb=load_workbook("D:/formula.xlsx")
ws=wb.active
#formula to find greater
ws["G5"]="=if(B5>B6,True)"
wb.save("D:/formula.xlsx")
```

Adding Styles/ formatting:

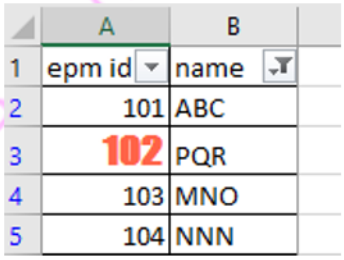
Styles are used to change the look of your data while displayed on screen. They are also used to determine the formatting for numbers.

Styles can be applied to the following aspects:
-font to set font size, color, underlining, etc.
-fill to set a pattern or color gradient
-border to set borders on a cell
-cell alignment
-Number Formatting

Font Formatting:

To format the font, we need to import Font class from openpyxl.styles.
The Font class provides all the functionality to format cell. Let us customize the font style, size, color, and more to enhance text appearance.

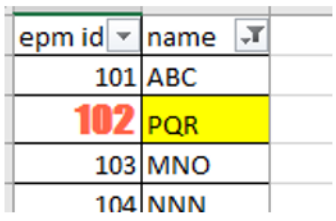
```
from openpyxl import load_workbook
from openpyxl.styles import Font
wb=load_workbook("D:/new.xlsx")
ws=wb.active
ws['A3'].font=Font(name="Impact",
                   bold=True,
              size=14,
              color="FF6347")
wb.save("D:/new.xlsx")
```



	A	B
1	epm id	name
2	101	ABC
3	102	PQR
4	103	MNO
5	104	NNN

fill to set a pattern or color gradient formatting:

```
from openpyxl import load_workbook
from openpyxl.styles import PatternFill
wb=load_workbook("D:/new.xlsx")
ws=wb.active
ws['B3'].fill=PatternFill(start_color="FFFF00",
                          end_color="FFFF00",
                          fill_type="solid")
wb.save("D:/new.xlsx")
```



epm id	name
101	ABC
102	PQR
103	MNO
104	NNN

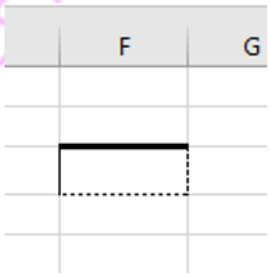
Where,
start_color="FFFF00":
This specifies the hexadecimal RGB value for the starting color of the fill. FFFF00 represents a bright yellow.

end_color="FFFF00":
This specifies the hexadecimal RGB value for the ending color of the fill. In this case, it's the same as the start_color, indicating a uniform color throughout the fill.
fill_type="solid":
This indicates that the fill should be a solid color, without any patterns or gradients.

borders on a cell:

We can modify borders of cells to define areas clearly by using the Border and Side classes from the openpyxl.styles as,

```
from openpyxl import load_workbook
from openpyxl.styles import Border, Side
wb=load_workbook("D:/new.xlsx")
ws=wb.active
border_thick = Side(style='thick')
border_thin = Side(style='thin')
borded_dashed = Side(style='dashed')
border_dotted = Side(style='dotted')

# Apply different border styles
ws['f3'].border = Border(top=border_thick,
                         left=border_thin,
                         right=borded_dashed,
                         bottom=border_dotted)
wb.save("D:/new.xlsx")
```



Cell alignment:

We can adjust the alignment of the text within cells using the Alignment class from openpyxl.styles.

```
from openpyxl import load_workbook
from openpyxl.styles import Alignment
wb=load_workbook("D:/new.xlsx")
ws=wb.active
ws['A4'].alignment =
Alignment(horizontal='center',
          vertical='center')
wb.save("D:/new.xlsx")
```

epm id	name
101	ABC
102	PQR
103	MNO

Number Formatting

We can apply number formats for displaying financial figures, dates, or other formatted numbers.

```
from openpyxl import load_workbook
from openpyxl.styles import Alignment
wb=load_workbook("D:/new.xlsx")
ws=wb.active
ws['F6'] = 14342345.6779
ws['F6'].number_format = '#,##0.00'

ws['F7'] = "2024-01-01"
ws['F7'].number_format = 'yyyy-mm-dd'
wb.save("D:/new.xlsx")
```

14,342,345.68
2024-01-01

Conditional Formatting:

While working with Excel files, conditional formatting is useful for the visualization of trends in the data, highlighting crucial data points, and making data more meaningful and understandable.

Conditional Formatting Rules:

CellIsRule: This rule indicates that we have to format cells based on the comparison between cell values and a specific condition, for example, greater than, less than, and more.

ColorScaleRule: This rule indicates that we can apply a color scale to a range of cells, where the color represents the relative value of the data.

FormulaRule: This rule involves applying formatting based on the result of a formula.

IconSetRule: This rule involves displaying icons in the cells based on the values of the cells.

10	20	30	40
15	25	35	45
50	60	70	80
5	10	15	20

```
# Importing openpyxl library
from openpyxl import Workbook
from openpyxl.styles import PatternFill
from openpyxl.formatting.rule import CellIsRule, ColorScaleRule, FormulaRule
# Create a workbook and select the active sheet
wb = Workbook()
ws = wb.active
# Add some sample data
data = [
    [10, 20, 30, 40],
    [15, 25, 35, 45],
    [50, 60, 70, 80],
    [5, 10, 15, 20]
]
for row in data:
    ws.append(row)
# Apply conditional formatting based on cell values
# Example 1: Highlight cells greater than 40 in red
red_fill = PatternFill(start_color="FFEE1110",
                       end_color="FFEE1101", fill_type="solid")
ws.conditional_formatting.add('A1:D4', CellIsRule(
    operator='greaterThan', formula=['40'], stopIfTrue=True, fill=red_fill))

# Example 2: Apply a 2-color scale between 10 and 80
ws.conditional_formatting.add('A1:D4', ColorScaleRule(start_type='num',
                              start_value=10, start_color='FFFFEDA0',
                              end_type='num', end_value=80,
                              end_color='FF00BFFF'))

# Example 3: Formula-based conditional formatting (highlight even numbers in
blue)
blue_fill = PatternFill(start_color="FF0000FF",
                        end_color="FF0000FF", fill_type="solid")
ws.conditional_formatting.add('A1:D4', FormulaRule(
    formula=['MOD(A1,2)=0'], fill=blue_fill))
# Save the workbook
wb.save('D:/co.xlsx')
```

Adding Images into Spreadsheet:

We can add images into spreadsheet for that **Pillow (PIL Fork)** MODULE is used.

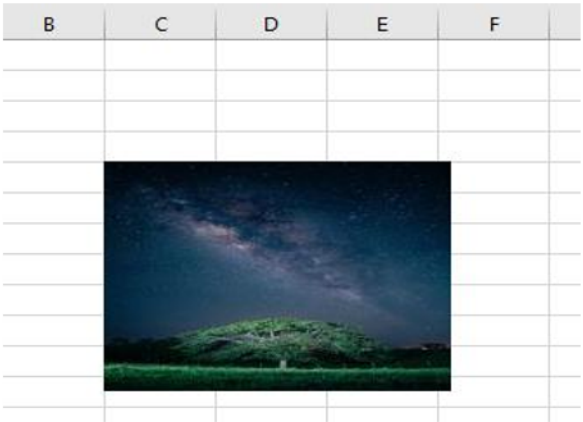
To add images in spreadsheet using Python, we need to combine Pillow with openpyxl.

First we have to install pillow by command prompt as,

```
pip install openpyxl Pillow
```

Example:

```
from PIL import Image
from openpyxl import Workbook
from openpyxl.drawing.image import Image as ExcelImage
# Renamed to avoid conflict with PIL.Image
# Open an image using Pillow
img_pil = Image.open("D:/im.jpg")
# Optional: Resize the image
img_pil = img_pil.resize((200, 150))
# Save the image temporarily or use the original path
img_pil.save("resized_image.jpg")
# Create a new workbook and select the active sheet
wb = Workbook()
ws = wb.active
# Create an openpyxl Image object from the image file
img_excel = ExcelImage("resized_image.jpg") # Or "your_image.png" if not resized
# Add the image to the specified cell (e.g., 'C5')
ws.add_image(img_excel, 'C5')
# Save the workbook
wb.save("d:/imgxl.xlsx")
```



Adding Charts:

Charts are a powerful way to visualize the data and making it easy to analyze and present information.

While working with Excel files in Python, the openpyxl we can represent data by creating charts like Barchart, Linechart, Pie-chart.

Some chart Terminologies:

Chart Data Series: A chart is usually comprised of one or more data series, out of which each series represents a collection of related data points that are plotted on the chart.

Chart Axes: As we know, charts often have two two axes X-axis (horizontal) and the Y-axis (vertical), these axes are used to map the data series to the chart area.

Anchoring the Chart: The charts must be positioned within the worksheet in our workbook. This can be done by specifying the cell where the top left corner of the chart should be anchored.

Steps to add charts:

1)Import necessary modules.

2)Create a workbook and worksheet.

3)Add data to the worksheet:Create a Reference object which defines the cell range containing the data to be charted. Specify the worksheet and the minimum/maximum rows and columns.

4)Create a chart object as BarChart() or LineChart(), to create a chart.

5)Add data to the chart by using add_data() method Use titles_from_data=True to automatically use the first row of your reference as the series title.

6)Add the chart to the worksheet by using the add_chart() method.

7)Save the workbook.

Let's create charts by using openpyxl

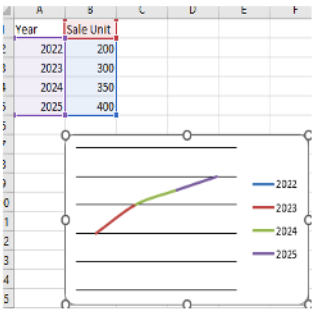
1) Bar Chart: Bar charts enable us to compare numerical values like integers and percentages. They use the length of each bar to represent the value of each variable.

Note: Reference objects link the chart to specific cell ranges in the worksheet.

```
# import necessary library
from openpyxl import Workbook
from openpyxl.chart import BarChart, Reference

# Create a new Workbook
wb = Workbook()
ws = wb.active

# Add data to the worksheet
data = [
    ["Year", "Sale Unit"],
    [2022, 200],
    [2023, 300],
    [2024, 350],
    [2025, 400],
]
for row in data:
    ws.append(row)
# Create a BarChart object
chart = BarChart()

# Define the data (sales values) for the chart
data = Reference(ws, min_col=2, min_row=1, max_row=5)

# Define the categories (years)
categories = Reference(ws, min_col=1, min_row=2, max_row=5)

# Add data and categories to the chart
chart.add_data(data, titles_from_data=True)
chart.set_categories(categories)

# Add the chart to the worksheet
ws.add_chart(chart, "B7")
# Save the Workbook to a file
wb.save("d:/sales.xlsx")
```

2) Line chart: A line chart (or line graph) is a type of chart that displays information as a series of data points connected by straight line segments

```
from openpyxl import Workbook
from openpyxl.chart import LineChart, Reference
wb = Workbook()
ws = wb.active
# Add data to the worksheet
data = [
    ["Year", "Unit"],
    [2020, 300],
    [2021, 400],
    [2022, 350],
    [2023, 500],
]
for row in data:
    ws.append(row)
# Create a LineChart object
chart = LineChart()

# Define the data (sales values) for the chart
data = Reference(ws, min_col=2, min_row=1,max_row=5)

# Define the categories (years)
categories = Reference(ws, min_col=1,min_row=2,max_row=5)


# Add data and categories to the chart
chart.add_data(data, titles_from_data=True)
chart.set_categories(categories)

# Add the chart to the worksheet
ws.add_chart(chart, "B7")
wb.save("d:/Line.xlsx")
```



3) Pie chart: A pie chart is a circular graph divided into "slices" to visually represent the proportional part-to-whole relationship of data.

```
from openpyxl import Workbook
from openpyxl.chart import PieChart, Reference
wb = Workbook()
ws = wb.active
# Add data to the worksheet
data = [
    ["Year", "Unit"],
    [2020, 300],
    [2021, 400],
    [2022, 350],
    [2023, 500],
]
for row in data:
    ws.append(row)
# Create a PieChart object
chart = PieChart()

# Define the data (sales values) for the chart
data = Reference(ws, min_col=2, min_row=1,max_row=5)

# Define the categories (years)
categories = Reference(ws, min_col=1,min_row=2,max_row=5)

# Add data and categories to the chart
chart.add_data(data, titles_from_data=True)
chart.set_categories(categories)

# Add the chart to the worksheet
ws.add_chart(chart, "B7")
wb.save("d:/Pie.xlsx")
```